

7/07/2025

Como

MANUALE TECNICO THE KNIFE

Viganò Matteo 760537

Vecaj Fabio 761232

INDICE

1.0 – TheKnife – Avvio dell'applicazione grafica

- 1.1 Descrizione generale
- 1.2 Obiettivi funzionali
- 1.3 Dipendenze e librerie utilizzate
- 1.4 Flusso di esecuzione
- 1.5 Posizionamento nel sistema
- 1.6 Responsabilità specifiche
- 1.7 Integrazione con il sistema

2.0 – GestoreFile – Gestione persistente dei dati su file CSV

- 2.1 Descrizione generale
- 2.2 Obiettivi funzionali
- 2.3 Dipendenze e librerie utilizzate
- 2.4 Gestione utenti
- 2.5 Gestione ristoranti
- 2.6 Gestione recensioni
- 2.7 Gestione preferiti
- 2.8 Ricerca ristoranti
- 2.9 Calcolo della media recensioni
- 2.10 Considerazioni progettuali

3.0 – RegisterDialog – Interfaccia grafica per la registrazione utenti

- 3.1 Descrizione generale
- 3.2 Obiettivi funzionali
- 3.3 Componenti GUI
- 3.4 Ciclo di interazione
- 3.5 Cifratura della password
- 3.6 Dipendenze e librerie
- 3.7 Considerazioni progettuali

4.0 – LoginDialog – Autenticazione utenti

- 4.1 Descrizione generale
- 4.2 Obiettivi funzionali
- 4.3 Componenti dell'interfaccia
- 4.4 Logica del login
- 4.5 Interazione con RegisterDialog
- 4.6 Dipendenze

5.0 – Utente – Modello dati per identità utente

- 5.1 Descrizione generale
- 5.2 Attributi
- 5.3 Costruttori
- 5.4 Metodi principali
- 5.5 Considerazioni progettuali
- 5.6 Utilizzo nel sistema

6.0 – Ristorante – Modello dati per entità ristorante

- 6.1 Descrizione generale
- 6.2 Attributi
- 6.3 Costruttori
- 6.4 Metodi principali
- 6.5 Utilizzo nel sistema

7.0 – Recensione – Modello dati per recensioni utenti

- 7.1 Descrizione generale
- 7.2 Attributi
- 7.3 Costruttori
- 7.4 Metodi principali
- 7.5 Validazioni
- 7.6 Utilizzo nel sistema

8.0 – GradientPanel – Pannello grafico con sfumatura

- 8.1 Descrizione generale
- 8.2 Attributi
- 8.3 Costruttore
- 8.4 Metodo paintComponent
- 8.5 Utilizzo nel sistema
- 8.6 Dipendenze
- 8.7 Considerazioni progettuali

9.0 – GradientMenuBar – Barra dei menu con gradiente grafico

- 9.1 Descrizione generale
- 9.2 Attributi
- 9.3 Costruttore
- 9.4 Metodo paintComponent
- 9.5 Utilizzo nel sistema
- 9.6 Dipendenze
- 9.7 Considerazioni progettuali

10.0 – FancyFrame – Finestra principale dell'applicazione

- 10.1 Descrizione generale
- 10.2 Attributi principali
- 10.3 Componenti grafici
- 10.4 Comportamento dinamico
- 10.5 Metodi principali
- 10.6 Costanti di navigazione
- 10.7 Dipendenze
- 10.8 Considerazioni progettuali

11.0 – SearchPanel – Interfaccia per la ricerca dei ristoranti

- 11.1 Descrizione generale
- 11.2 Obiettivi funzionali
- 11.3 Componenti dell'interfaccia
- 11.4 Funzionamento della ricerca
- 11.5 Metodo refresh()
- 11.6 Dipendenze
- 11.7 Considerazioni progettuali

12.0 – RistorantiPanel – Gestione dei ristoranti del ristorante

- 12.1 Descrizione generale
- 12.2 Obiettivi funzionali
- 12.3 Componenti principali
- 12.4 Metodo refreshData()
- 12.5 Metodo mostraDialogNuovoRistorante()
- 12.6 Dipendenze
- 12.7 Considerazioni progettuali

13.0 – RestaurantDetailPanel – Visualizzazione dettagliata del ristorante

- 13.1 Descrizione generale
- 13.2 Obiettivi funzionali
- 13.3 Componenti GUI
- 13.4 Metodo setRistorante
- 13.5 Metodo refreshDetails()
- 13.6 Dipendenze
- 13.7 Considerazioni progettuali

14.0 – RecensioniPanel – Gestione delle recensioni utenti

- 14.1 Descrizione generale
- 14.2 Obiettivi funzionali
- 14.3 Componenti GUI
- 14.4 Metodo refreshData()
- 14.5 Metodo nuovaRecensione()
- 14.6 Metodo modificaRecensione()
- 14.7 Metodo eliminaRecensione()
- 14.8 Metodo rispondiRecensione()
- 14.9 Considerazioni progettuali

14.10 Dipendenze

15.0 – PreferitiPanel – Gestione dei ristoranti preferiti del cliente

- 15.1 Descrizione generale
- 15.2 Obiettivi funzionali
- 15.3 Componenti GUI
- 15.4 Metodo refreshData()
- 15.5 Metodo aggiungiPreferito()
- 15.6 Metodo rimuoviPreferito()
- 15.7 Dipendenze
- 15.8 Considerazioni progettuali

16.0 – HomePanel – Pannello di benvenuto

- 16.1 Descrizione generale
- 16.2 Obiettivi funzionali
- 16.3 Componenti GUI
- 16.4 Logica costruttiva
- 16.5 Dipendenze
- 16.6 Considerazioni progettuali

1.0 - Classe *TheKnife* – Avvio dell'applicazione grafica

1.1 Descrizione generale

La classe *TheKnife*, situata nel package `theknife.gui`, rappresenta il punto di ingresso dell'intera applicazione. Viene eseguita automaticamente al momento dell'avvio del programma ed è responsabile della corretta inizializzazione dell'interfaccia grafica utente (GUI), realizzata con la libreria Java Swing.

Il suo scopo primario è avviare la finestra principale dell'applicazione, applicando prima un tema grafico coerente (look and feel) e successivamente costruendo e visualizzando la GUI su un thread sicuro e dedicato alla gestione degli eventi grafici.

1.2 Obiettivi funzionali

La classe *TheKnife* svolge le seguenti funzioni principali:

Applicazione del tema grafico personalizzato Nimbus a tutta l'interfaccia.

Costruzione della finestra principale dell'applicazione (*FancyFrame*).

Esecuzione della GUI sul thread dedicato agli eventi grafici (Event Dispatch Thread) mediante l'utilizzo della funzione `invokeLater` della libreria Swing.

La classe non gestisce dati, logica di business né interazioni dirette con l'utente. Ha esclusivamente un ruolo tecnico di inizializzazione.

1.3 Dipendenze e librerie utilizzate

<u>Componente</u>	<u>Descrizione</u>
-------------------	--------------------

<i>javax.swing.SwingUtilities</i>	Libreria standard Java Swing per la gestione sicura dei thread grafici.
-----------------------------------	---

<i>FancyFrame</i>	Classe che rappresenta la finestra principale dell'interfaccia grafica.
-------------------	---

L'utilizzo di *SwingUtilities* è essenziale per assicurare che le operazioni grafiche vengano eseguite sul thread corretto, evitando errori di concorrenza o aggiornamenti non sincronizzati dell'interfaccia.

1.4 Flusso di esecuzione

Al momento dell'avvio dell'applicazione, la JVM esegue il metodo `main()` contenuto in questa classe. I passaggi seguiti sono:

Richiamo al metodo `setCustomNimbusLookAndFeel()` della classe *FancyFrame*, che applica il tema grafico Nimbus personalizzato all'interfaccia.

Utilizzo di `SwingUtilities.invokeLater()` per garantire che l'interfaccia venga costruita ed eseguita sul thread grafico dedicato (EDT).

Creazione di un'istanza della classe *FancyFrame* (la finestra principale).

Visualizzazione della finestra principale all'utente con `setVisible(true)`.

Questo flusso garantisce che la GUI venga avviata correttamente, in modo sicuro e conforme agli standard Swing.

1.5 Posizionamento nel sistema

Package: `theknife.gui`

Classe avviata: *FancyFrame*

Funzione del metodo `main`: punto di avvio della GUI

Thread grafico utilizzato: EDT (Event Dispatch Thread)

Tema applicato: Nimbus personalizzato (gestito esternamente)

1.6 Responsabilità specifiche

Ambito Responsabilità della classe ***TheKnife***

Inizializzazione grafica Applicare un look and feel coerente a tutta la GUI

Creazione della GUI Costruire la finestra principale e avviarla

Sicurezza di esecuzione Garantire che tutto avvenga sul thread EDT

Separazione della logica Non contiene alcuna logica applicativa

1.7 Integrazione con il sistema

La classe ***TheKnife*** non gestisce componenti di logica, non elabora dati e non si occupa di interazioni con file o database. Essa dipende unicamente dalla classe `FancyFrame`, la quale contiene e visualizza tutti i pannelli, i menu e le funzionalità dell'applicazione.

La progettazione di questa classe segue il principio della singola responsabilità (Single Responsibility Principle), isolando il bootstrap dell'interfaccia dal resto del sistema. Qualsiasi modifica o estensione del flusso di avvio deve essere implementata in questa classe.

2.0 - Classe `GestoreFile` – Gestione persistente dei dati su file CSV

2.1 Descrizione generale

La classe `GestoreFile` è una classe di servizio centrale per la gestione dell'interazione tra il sistema e i file di dati esterni. Essa gestisce la ****lettura e la scrittura su file CSV**** per tutte le entità principali dell'applicazione: utenti, ristoranti, recensioni, preferiti e risultati di ricerca. Tutte le operazioni sono orientate alla persistenza dei dati, permettendo al sistema di memorizzare e recuperare informazioni tra una sessione e l'altra. L'approccio seguito è interamente basato su file flat (formato CSV) e sfrutta la libreria di base `java.io` per operazioni di I/O.

2.2 Obiettivi funzionali

La classe offre un insieme di metodi statici suddivisi per categoria, con i seguenti scopi principali:

- * Caricamento e salvataggio di utenti da/verso un file CSV.
- * Caricamento e salvataggio di ristoranti da/verso un file CSV.
- * Aggiunta, modifica ed eliminazione di recensioni persistenti.
- * Gestione dei ristoranti preferiti da parte degli utenti.
- * Ricerca filtrata di ristoranti secondo più criteri.
- * Calcolo della media delle valutazioni per un ristorante.

La classe non istanzia oggetti propri e non mantiene stato: tutte le sue operazioni sono ****stateless**** e statiche.

2.3 Dipendenze e librerie utilizzate

Libreria	Scopo
java.io.*	Lettura e scrittura di file CSV tramite classi come <code>BufferedReader</code> , <code>FileReader</code> , <code>FileWriter</code> , <code>PrintWriter</code> .
java.util.*	Gestione di strutture dati come <code>List</code> , <code>ArrayList</code> , <code>Map</code> , ecc.
java.util.stream.*	Utilizzo degli stream per effettuare operazioni su collezioni come filtri, mappature e ricerche.

2.4 Gestione utenti

I metodi `caricaUtenti` e `salvaUtenti` leggono e scrivono l'elenco utenti in file CSV con 7 campi: nome, cognome, username, password cifrata, data di nascita, domicilio e ruolo.

- * Il metodo `usernameEsistente` verifica la presenza di un determinato username in un file esistente.
- * Il metodo `aggiungiUtente` carica tutti gli utenti, verifica che lo username non esista già, e in tal caso aggiunge il nuovo utente e salva l'elenco.

2.5 Gestione ristoranti

La sezione relativa ai ristoranti opera su file con 11 colonne, tra cui: nome, nazione, città, indirizzo, coordinate geografiche, fascia di prezzo, opzioni di consegna/prenotazione, tipo di cucina e proprietario.

- * `caricaRistoranti` legge il file CSV e costruisce la lista di oggetti `Ristorante`.
- * `salvaRistoranti` salva una lista di ristoranti sovrascrivendo il file.
- * `aggiungiRistorante` evita i duplicati controllando se esiste già un ristorante con lo stesso nome.

2.6 Gestione recensioni

Le recensioni sono gestite con metodi che leggono e scrivono file CSV con 5 campi: nome ristorante, username, stelle (valutazione 1-5), testo della recensione e risposta del ristoratore.

- * ``caricaRecensioni`` legge le recensioni e normalizza i campi, anche quando il campo "risposta" è vuoto.
- * ``salvaRecensioni`` sovrascrive l'intero file delle recensioni.
- * ``aggiungiRecensione`` aggiunge una nuova riga al file senza cancellare quelle esistenti.
- * ``modificaRecensione`` cerca una recensione specifica (utente + ristorante) e ne aggiorna stelle e testo.
- * ``eliminaRecensione`` rimuove una recensione dal file in base all'utente e al ristorante.

2.7 Gestione preferiti

I preferiti sono rappresentati come coppie ``username`` - ``nome ristorante``. I metodi disponibili permettono:

- * ``caricaPreferiti``: restituisce una lista dei ristoranti preferiti per un utente specifico.
- * ``salvaPreferiti``: sovrascrive completamente il file.
- * ``aggiungiPreferito``: aggiunge una coppia solo se non è già presente.
- * ``rimuoviPreferito``: elimina una coppia corrispondente dal file.

2.8 Ricerca ristoranti

Il metodo ``cercaRistoranti`` consente una ricerca filtrata dei ristoranti a partire da più criteri opzionali:

- * Nome (anche parziale)
- * Città
- * Tipo di cucina
- * Prezzo massimo
- * Disponibilità di delivery
- * Disponibilità di prenotazione

Il metodo applica ogni filtro solo se il valore passato è non nullo e non vuoto. Il risultato è una lista di ristoranti che soddisfano ****tutti**** i criteri attivati.

2.9 Calcolo della media recensioni

Il metodo ``calcolaMediaStelle`` calcola la media aritmetica delle stelle di tutte le recensioni relative a un ristorante, ignorando i ristoranti senza recensioni. Il risultato viene restituito come valore ``double``, con precisione standard.

2.10 Considerazioni progettuali

La classe ``GestoreFile`` è progettata per mantenere le operazioni I/O isolate e riutilizzabili. L'uso esclusivo di metodi statici elimina la necessità di istanziare la classe e rende il codice più semplice e centralizzato. Il formato CSV è una scelta accessibile, compatibile con strumenti esterni (come Excel), ma richiede attenzione alla formattazione e alla gestione dei campi vuoti.

La struttura a sezioni (utenti, ristoranti, recensioni, preferiti, ricerca) riflette chiaramente i moduli logici dell'applicazione.

3.0 - Classe `RegisterDialog` – Interfaccia grafica per la registrazione utenti

3.1 Descrizione generale

La classe `RegisterDialog` rappresenta una finestra modale di dialogo per la registrazione di nuovi utenti all'interno del sistema. Essa è costruita mediante l'utilizzo delle componenti di **Swing** ed estende `JDialog`, permettendo all'interfaccia di apparire in primo piano rispetto alla finestra chiamante.

Tale classe consente agli utenti di inserire i propri dati (nome, cognome, username, password e ruolo) per creare un nuovo account, ed effettuare validazioni, cifratura della password e salvataggio dei dati su file.

3.2 Obiettivi funzionali

Gli obiettivi principali della classe sono:

- * Fornire un'interfaccia grafica per la compilazione dei dati anagrafici necessari alla registrazione.
- * Validare i dati inseriti assicurandosi che i campi obbligatori siano compilati.
- * Cifrare la password inserita utilizzando l'algoritmo SHA-256.
- * Verificare che lo username non sia già presente nei dati esistenti.
- * Creare un oggetto `Utente` e salvarlo tramite `GestoreFile`.

3.3 Componenti GUI

La finestra modale include i seguenti elementi:

- * Campi di testo per **nome**, **cognome**, **username**.
- * Campo password per l'inserimento in sicurezza della password.
- * ComboBox per la selezione del ruolo (cliente o ristoratore).
- * Pulsanti OK e Annulla.

La disposizione è gestita con un layout `GridBagLayout`, che consente un controllo preciso sul posizionamento dei componenti.

3.4 Ciclo di interazione

Alla pressione del pulsante **OK**, il programma:

1. Recupera i dati inseriti nei campi.
2. Controlla che nessun campo obbligatorio sia vuoto.
3. Verifica l'unicità dello username attraverso `GestoreFile.usernameEsistente()`.
4. Applica la cifratura SHA-256 alla password tramite la funzione interna `cifraPassword`.
5. Istanza un oggetto `Utente` con i dati raccolti.
6. Salva il nuovo utente nel file `data/utenti.csv` tramite `GestoreFile.aggiungiUtente()`.
7. Mostra un messaggio di successo e chiude la finestra.

Alla pressione del pulsante **Annulla**, la finestra si chiude senza eseguire alcuna operazione.

3.5 Cifratura della password

La funzione privata `cifraPassword(String password)` implementa la cifratura mediante algoritmo **SHA-256**. Utilizza la classe `MessageDigest` della libreria `java.security` e converte la password in una stringa esadecimale. Questo garantisce che la password non venga mai salvata in chiaro nei file.

3.6 Dipendenze e librerie

Libreria	Scopo
javax.swing.*	Costruzione e gestione dell'interfaccia grafica.
java.awt.*	Gestione dei layout e componenti grafici avanzati.
java.security.MessageDigest	Cifratura sicura delle password tramite hashing SHA-256.
theknife.GestoreFile	Gestione della lettura e scrittura su file CSV del sistema.
theknife.Utente	Definizione e utilizzo dell'entità utente nella registrazione.

3.7 Considerazioni progettuali

La classe segue un modello ****self-contained****, in cui tutta la logica di validazione, verifica e cifratura è contenuta internamente. Questo semplifica l'integrazione con il resto del sistema e migliora la leggibilità del codice.

L'utilizzo di un `JDialog` modale è una scelta progettuale corretta per evitare che l'utente interagisca con altre parti del programma finché non ha completato (o annullato) la registrazione. La cifratura della password rappresenta una buona pratica di sicurezza per proteggere i dati degli utenti, pur in un contesto senza database.

4.0 - Classe `LoginDialog` – Autenticazione utenti

4.1 Descrizione generale

La classe `LoginDialog` gestisce la finestra di accesso degli utenti. Anch'essa estende `JDialog` e fornisce un'interfaccia grafica per inserire le credenziali di login (username, password) e selezionare il ruolo (`cliente` o `ristoratore`).

L'interfaccia permette anche la registrazione di un nuovo utente grazie a un pulsante che apre un `RegisterDialog`.

4.2 Obiettivi funzionali

- * Permettere all'utente di autenticarsi tramite username e password.
- * Verificare che il ruolo selezionato corrisponda a quello associato all'utente.
- * In caso di credenziali errate, mostrare un messaggio di errore.
- * Consentire la registrazione tramite dialog secondario.

4.3 Componenti dell'interfaccia

- * Campo di testo per `username`
- * Campo `JPasswordField` per la password
- * ComboBox per la selezione del ruolo
- * Pulsanti: Login, Cancel, Registrati

La disposizione è curata tramite `GridBagLayout`.

4.4 Logica del login

Alla pressione del pulsante **Login**, il metodo `effettuaLogin`:

1. Carica gli utenti da file usando `GestoreFile.caricaUtenti()`.
2. Confronta l'username (ignorando il case), la password cifrata e il ruolo.
3. Se la corrispondenza è trovata, restituisce l'oggetto `Utente`.
4. Se nessuna corrispondenza è trovata, mostra un errore.

La password viene cifrata al momento del confronto tramite la funzione `cifraPassword`, che utilizza l'algoritmo SHA-256.

4.5 Interazione con `RegisterDialog`

Alla pressione del pulsante **Registrati**, viene istanziato e mostrato un `RegisterDialog`. Questo permette all'utente, in qualsiasi momento, di creare un nuovo account.

4.6 Dipendenze

Classe	Scopo
theknife.GestoreFile	Lettura e scrittura dei dati utente e altre entità da/verso file.
theknife.Utente	Rappresenta un utente del sistema con attributi e ruolo.
RegisterDialog	Finestra per la registrazione di un nuovo utente.

5.0 - Classe `Utente` – Modello dati per identità utente

5.1 Descrizione generale

La classe `Utente` rappresenta il modello dati degli utenti del sistema. Ogni oggetto `Utente` contiene tutte le informazioni anagrafiche e di autenticazione necessarie per identificare un utente, sia esso un cliente o un ristoratore.

La classe è progettata per essere semplice, immagazzinando i dati principali e fornendo metodi di accesso (getter/setter), oltre a due metodi di utilità per distinguere il tipo di ruolo.

5.2 Attributi

Attributo	Tipo	Descrizione
nome	String	Nome dell'utente.
cognome	String	Cognome dell'utente.
username	String	Identificativo unico dell'utente nel sistema.
passwordCifrata	String	Password dell'utente cifrata tramite SHA-256.
dataNascita	String	Data di nascita dell'utente.
domicilio	String	Indirizzo di domicilio dell'utente.
ruolo	String	Ruolo dell'utente: "cliente" o "ristoratore".

5.3 Costruttori

- * Costruttore vuoto: necessario per l'inizializzazione flessibile.
- * Costruttore completo: accetta tutti i campi come parametri.

5.4 Metodi principali

- * Metodi getter e setter per ogni attributo, per accedere e modificare i valori.
- * `isCliente()`: restituisce `true` se il ruolo è "cliente".
- * `isRistoratore()`: restituisce `true` se il ruolo è "ristoratore".
- * `toString()`: restituisce una rappresentazione testuale dell'oggetto, utile per debug o log.

5.5 Considerazioni progettuali

Questa classe funge da semplice contenitore di dati e non implementa alcuna logica applicativa. È serializzabile in CSV tramite la classe `GestoreFile` e permette una distinzione immediata dei ruoli grazie ai metodi `isCliente()` e `isRistoratore()`.

La scelta di mantenere la password cifrata fin dall'origine rafforza la sicurezza delle credenziali, evitando qualsiasi esposizione del dato sensibile in chiaro.

5.6 Utilizzo nel sistema

La classe `Utente` è centrale e viene utilizzata:

- * Nei processi di login e registrazione (`LoginDialog`, `RegisterDialog`).
- * Nella validazione dei dati utente.
- * Per determinare permessi e visibilità a livello di interfaccia e funzionalità.
- * Per salvare/caricare dati dal file `utenti.csv`.

6.0 - Classe `Ristorante` – Modello dati per entità ristorante

6.1 Descrizione generale

La classe `Ristorante` modella l'entità ristorante nel sistema. Ogni oggetto rappresenta un locale con tutte le caratteristiche necessarie per essere cercato, recensito e gestito da un ristorante. Il modello prevede 11 campi, incluso il nome del proprietario (ristoratore) che rappresenta l'associazione diretta tra ristorante e utente.

6.2 Attributi

Attributo	Tipo	Descrizione
nome	String	Nome del ristorante.
nazione	String	Nazione in cui si trova il ristorante.
città	String	Città del ristorante.
indirizzo	String	Indirizzo fisico del locale.
latitudine	double	Coordinata geografica di latitudine.
longitudine	double	Coordinata geografica di longitudine.
fasciaPrezzo	double	Fascia di prezzo espressa come valore numerico.
delivery	boolean	Disponibilità di servizio a domicilio.
prenotazione	boolean	Possibilità di prenotare un tavolo.
tipoCucina	String	Tipo di cucina offerta (es. "italiana", "giapponese").
proprietario	String	Username del ristorante proprietario del locale.

6.3 Costruttori

- * Costruttore vuoto: utile per l'inizializzazione da framework o deserializzazione.
- * Costruttore completo: accetta tutti gli 11 campi per l'istanziamento.

6.4 Metodi principali

- * Getter e setter per ogni attributo.
- * Metodo `toString()` per restituire una rappresentazione testuale leggibile del ristorante.

6.5 Utilizzo nel sistema

La classe `Ristorante` è utilizzata:

- * Nella ricerca dei ristoranti tramite filtri.
- * Per l'associazione al proprietario (ristoratore).
- * Per il caricamento e salvataggio dei dati da/verso file CSV.
- * Come entità visualizzata nell'interfaccia utente e nei dettagli.

7.0 - Classe Recensione – Modello dati per recensioni utenti

7.1 Descrizione generale

La classe Recensione rappresenta l'entità recensione nel sistema. Ogni oggetto Recensione raccoglie le informazioni fornite da un cliente riguardo un ristorante specifico, includendo anche l'eventuale risposta da parte del ristorante.

L'oggetto è progettato per essere semplice e completamente serializzabile in formato CSV. Questo consente l'integrazione con la persistenza file, utile per leggere e scrivere recensioni su disco senza l'uso di database.

7.2 Attributi

Attributo	Tipo	Descrizione
idRistorante	String	Nome identificativo del ristorante oggetto della recensione.
username	String	Username dell'utente autore della recensione.
stelle	int	Voto espresso in stelle (valori ammessi da 1 a 5).
testo	String	Testo libero che descrive l'opinione del cliente.
risposta	String	Eventuale replica del ristorante (può essere vuota o null).

7.3 Costruttori

Costruttore vuoto: richiesto per eventuali operazioni di deserializzazione e inizializzazione flessibile.

Costruttore parametrizzato: inizializza tutti i campi, includendo una validazione sul campo stelle.

7.4 Metodi principali

Getter/Setter: per tutti i campi, permettono di accedere e modificare ciascun attributo.

setStelle(int stelle): applica una validazione obbligatoria, lanciando un'eccezione se il valore non è compreso tra 1 e 5.

hasRisposta(): restituisce true se il campo risposta è valorizzato e non vuoto.

toString(): fornisce una rappresentazione testuale utile per il debug e il logging.

7.5 Validazioni

La validazione più rilevante è applicata all'attributo stelle, dove viene imposto un vincolo numerico per garantire coerenza semantica. In caso di valori non validi, viene lanciata un'eccezione `IllegalArgumentException`.

7.6 Utilizzo nel sistema

La classe Recensione è utilizzata:

Nella fase di scrittura, modifica ed eliminazione delle recensioni da parte dei clienti.

Per la visualizzazione all'interno dell'interfaccia utente (in tabella o dettaglio).

Per il calcolo della media delle stelle nei ristoranti (`GestoreFile.calcolaMediaStelle`).

Per consentire la risposta da parte del ristorante.

8.0 - Classe GradientPanel – Pannello grafico con sfumatura

8.1 Descrizione generale

La classe GradientPanel estende JPanel e fornisce una personalizzazione grafica avanzata per la visualizzazione di uno sfondo con gradiente verticale. Questo componente è progettato per essere riutilizzabile all'interno dell'interfaccia grafica, migliorando l'aspetto visivo di finestre e pannelli.

Grazie all'utilizzo della classe GradientPaint di Java AWT, consente di definire una transizione fluida tra due colori specificati in fase di costruzione. Questo effetto visivo è particolarmente utile per migliorare l'estetica dell'applicazione.

8.2 Attributi

Attributo	Tipo	Descrizione
startColor	Color	Colore iniziale del gradiente (posizione superiore del pannello).
endColor	Color	Colore finale del gradiente (posizione inferiore del pannello).

Entrambi gli attributi vengono inizializzati tramite il costruttore e definiscono l'effetto di sfumatura.

8.3 Costruttore

```
public GradientPanel(Color startColor, Color endColor)
```

Il costruttore accetta due parametri Color che rappresentano rispettivamente il colore di inizio e fine del gradiente. La chiamata a setOpaque(false) disattiva la pittura del background standard di JPanel, permettendo al metodo paintComponent di gestire completamente la resa grafica.

8.4 Metodo paintComponent(Graphics g)

Questo metodo è sovrascritto per disegnare manualmente il gradiente. Il processo avviene nel seguente ordine:

Creazione del contesto grafico (Graphics2D): viene effettuato il cast dell'oggetto Graphics per poter usare metodi avanzati.

Recupero delle dimensioni del pannello: attraverso getWidth() e getHeight().

Creazione dell'oggetto GradientPaint: che definisce il gradiente verticale dal colore startColor al colore endColor.

Applicazione del gradiente: riempiendo l'intera area del pannello.

Rilascio delle risorse grafiche: chiamando dispose() sull'oggetto Graphics2D.

Chiamata a super.paintComponent(g): per permettere ad altri componenti figlio (se presenti) di disegnarsi correttamente sopra.

8.5 Utilizzo nel sistema

Il GradientPanel può essere utilizzato ovunque sia richiesto uno sfondo visivamente gradevole, come:

Frame principali

Intestazione di sezione

Pannelli centrali di finestre modali

Viene spesso utilizzato per creare una coerenza visiva in combinazione con lo stile dell'applicazione, evitando sfondi statici e monocromatici.

8.6 Dipendenze

Classe/Package Scopo

javax.swing.JPanel Classe base per componenti grafici contenitori.

java.awt.* Per la grafica personalizzata e gestione colore.

8.7 Considerazioni progettuali

L'implementazione di GradientPanel è completamente incapsulata e può essere facilmente riutilizzata in qualsiasi parte del sistema. Il codice è indipendente da altri componenti e consente un alto grado di modularità. Inoltre, l'uso del gradiente contribuisce a rendere l'interfaccia più moderna e professionale.

Non introduce logica applicativa, ma si concentra esclusivamente sull'aspetto grafico, rispettando il principio della separazione delle responsabilità.

9.0 - Classe GradientMenuBar – Barra dei menu con gradiente grafico

9.1 Descrizione generale

La classe GradientMenuBar estende la classe standard JMenuBar di Swing, introducendo una personalizzazione grafica basata su un gradiente verticale. Questa barra dei menu visivamente migliorata è pensata per uniformare lo stile grafico dell'applicazione, in particolare per adattarsi visivamente ai componenti come GradientPanel.

La classe è progettata per essere utilizzata in modo intercambiabile con qualsiasi JMenuBar convenzionale, ma con un aspetto estetico decisamente più moderno e gradevole.

9.2 Attributi

Attributo	Tipo	Descrizione
startColor	Color	Colore iniziale del gradiente (parte superiore della barra).
endColor	Color	Colore finale del gradiente (parte inferiore della barra).

I colori del gradiente vengono definiti al momento dell'istanza e controllano l'estetica verticale della barra.

9.3 Costruttore

```
public GradientMenuBar(Color startColor, Color endColor)
```

Questo costruttore inizializza la barra dei menu con i due colori che determinano la sfumatura verticale. Inoltre:

Imposta setOpaque(false) per evitare la pittura predefinita di fondo della JMenuBar.

Imposta un font coerente e leggibile (Segoe UI, bold, dimensione 14pt) per le voci di menu.

9.4 Metodo paintComponent(Graphics g)

Il metodo paintComponent è sovrascritto per disegnare la sfumatura sull'intera superficie della barra dei menu. I passaggi sono:

Creazione di un contesto grafico tramite cast a Graphics2D.

Definizione delle dimensioni attuali del componente.

Creazione di un GradientPaint verticale dal colore iniziale a quello finale.

Riempimento dell'intera area con la sfumatura tramite fillRect().

Rilascio delle risorse grafiche con dispose().

Questo approccio consente una resa visiva di alta qualità senza influenzare la funzionalità dei menu Swing.

9.5 Utilizzo nel sistema

La classe GradientMenuBar può essere utilizzata come sostituto diretto di una JMenuBar standard nei frame dell'applicazione. È ideale per:

Finestra principale (FancyFrame)

Qualsiasi finestra che richieda una barra dei menu integrata visivamente con pannelli sfumati

3.13.6 Dipendenze

Classe/Package	Scopo
----------------	-------

javax.swing.*	Gestione dell'interfaccia grafica Swing.
---------------	--

java.awt.*	Funzionalità grafiche di basso livello, incluse Color e Font.
------------	---

9.7 Considerazioni progettuali

GradientMenuBar è un esempio di incapsulamento visivo: la classe non introduce logica di business ma si occupa unicamente della presentazione grafica. È completamente riutilizzabile, configurabile e può essere integrata in qualsiasi contesto dove si desideri migliorare l'estetica della barra dei menu.

L'impostazione del font direttamente nel costruttore contribuisce all'uniformità dello stile, evitando la necessità di configurazioni aggiuntive nei frame chiamanti.

10.0 - Classe FancyFrame – Finestra principale dell'applicazione

10.1 Descrizione generale

La classe FancyFrame rappresenta la finestra principale dell'applicazione TheKnife. Estende JFrame e integra l'interfaccia utente completa, composta da:

Barra superiore personalizzata con logo e titolo.

Barra dei menu (JMenuBar) con voci di navigazione.

Pannello centrale con layout a schede (CardLayout) che ospita i vari pannelli funzionali.

Logica di gestione login/logout e controllo del ruolo utente.

Tutte le funzionalità principali del sistema sono accessibili da questa finestra, rendendola il fulcro dell'interazione utente.

10.2 Attributi principali

Attributo	Tipo	Descrizione
cardLayout	CardLayout	Gestore delle viste a schede (card) per il pannello centrale.
centerPanel	JPanel	Contenitore dei pannelli con CardLayout.
utenteCorrente	Utente	Utente autenticato al momento.
homePanel	HomePanel	Pannello iniziale.
searchPanel	SearchPanel	Pannello per la ricerca dei ristoranti.
ristorantiPanel	RistorantiPanel	Pannello per la gestione dei ristoranti (solo per ristoratori).
recensioniPanel	RecensioniPanel	Pannello per la visualizzazione e gestione delle recensioni.
preferitiPanel	PreferitiPanel	Pannello che mostra i ristoranti preferiti del cliente.
detailPanel	RestaurantDetailPanel	Pannello dei dettagli di un ristorante selezionato.

10.3 Componenti grafici

Top Bar: barra superiore blu con logo a sinistra e titolo centrato.

JMenuBar: menu a tendina con le voci:

Home

Ricerca

Miei Ristoranti (visibile solo ai ristoratori autenticati)

Recensioni (disponibile solo agli utenti autenticati)

Preferiti (visibile solo ai clienti autenticati)

Login / Logout (in base allo stato attuale dell'utente)

Il look generale è coerente e personalizzato con colori aziendali e font "Segoe UI".

10.4 Comportamento dinamico

Al click su Login, si apre la finestra LoginDialog. Se l'autenticazione va a buon fine, l'utente viene salvato in utenteCorrente.

Se è già autenticato, il pulsante diventa Logout, e consente di terminare la sessione corrente.

Le voci del menu abilitano l'accesso a specifiche sezioni in base al ruolo:

Miei Ristoranti: accessibile solo ai ristoratori autenticati.

Preferiti: accessibile solo ai clienti autenticati.

Recensioni: disponibile a tutti gli utenti autenticati.

Le viste vengono cambiate dinamicamente tramite cardLayout.show(...) con il metodo showCard.

10.5 Metodi principali

Metodo	Descrizione
<code>showCard(String name)</code>	Cambia la vista mostrata nel <code>centerPanel</code> secondo l' identificatore dato.
<code>isLoggedIn()</code>	Restituisce <code>true</code> se un utente è autenticato.
<code>getUtenteCorrente()</code>	Ritorna l'oggetto <code>Utente</code> attualmente loggato.
<code>getDetailPanel()</code>	Restituisce il pannello dettagli ristorante per manipolazioni dinamiche.
<code>setCustomNimbusLookAndFeel()</code>	Imposta il look and feel Nimbus come stile predefinito.

10.6 Costanti di navigazione

Costante	Valore Stringa	Pannello associato
<code>CARD_HOME</code>	<code>"HomePanel"</code>	Pannello iniziale.
<code>CARD_SEARCH</code>	<code>"SearchPanel"</code>	Pannello di ricerca ristoranti.
<code>CARD_RISTORANTI</code>	<code>"RistorantiPanel"</code>	Pannello per i ristoranti gestiti.
<code>CARD_RECENSIONI</code>	<code>"RecensioniPanel"</code>	Pannello per gestione recensioni.
<code>CARD_PREFERITI</code>	<code>"PreferitiPanel"</code>	Pannello preferiti per clienti.
<code>CARD_DETAIL</code>	<code>"RestaurantDetailPanel"</code>	Dettaglio di un ristorante.

Queste costanti sono usate per semplificare la gestione delle schede nel `CardLayout`.

10.7 Dipendenze

Classe	Scopo
<code>javax.swing.*</code>	Costruzione dell'interfaccia grafica.
<code>java.awt.*</code>	Gestione layout e personalizzazione grafica.
<code>theknife.Utente</code>	Modello dell'utente autenticato.
<code>theknife.gui.panels.*</code>	Pannelli contenuti nella finestra principale.
<code>LoginDialog</code>	Dialog di autenticazione richiamato dal menu.

10.8 Considerazioni progettuali

La classe `FancyFrame` implementa il pattern card-based navigation usando `CardLayout`. Questo approccio è altamente modulare, poiché consente di aggiornare e gestire ogni pannello separatamente mantenendo un'interfaccia utente unificata.

L'integrazione con il sistema di login/log-out rende `FancyFrame` uno snodo centrale per la gestione dei permessi e delle funzionalità. L'uso di messaggi informativi tramite `JOptionPane` migliora l'esperienza utente, segnalando in tempo reale eventuali restrizioni.

11.0 - Classe SearchPanel – Interfaccia per la ricerca dei ristoranti

11.1 Descrizione generale

La classe SearchPanel rappresenta il pannello grafico per la ricerca avanzata di ristoranti nel sistema. Estende la classe personalizzata GradientPanel per applicare uno sfondo sfumato e fornisce un'interfaccia interattiva all'utente finale per filtrare i ristoranti in base a diversi criteri. Viene integrata nel frame principale (FancyFrame) e consente l'accesso ai dettagli del ristorante selezionato.

11.2 Obiettivi funzionali

Fornire un'interfaccia di ricerca flessibile per ristoranti.

Permettere l'inserimento di più criteri di filtro contemporaneamente.

Validare e processare il prezzo massimo inserito.

Visualizzare dinamicamente i risultati della ricerca sotto forma di pulsanti cliccabili.

Permettere la navigazione verso il pannello di dettaglio del ristorante selezionato.

11.3 Componenti dell'interfaccia

Componente	Tipo	Descrizione
txtNome	TextField	Campo per inserire il nome parziale del ristorante.
txtCitta	TextField	Campo per specificare la città.
txtCucina	TextField	Campo per specificare il tipo di cucina.
txtPrezzo	TextField	Campo per indicare il prezzo massimo desiderato.
chkDelivery	CheckBox	CheckBox per filtrare ristoranti con servizio di delivery.
chkPrenotazione	CheckBox	CheckBox per filtrare ristoranti con possibilità di prenotazione.
btnCerca	Button	Pulsante per eseguire la ricerca.
resultsPanel	JPanel	Pannello verticale che mostra i risultati della ricerca.

Il layout generale è suddiviso in:

Un pannello superiore con filtro.

Un pannello centrale scrollabile con i risultati.

11.4 Funzionamento della ricerca

Il metodo eseguiRicerca() viene invocato al click sul pulsante "Cerca" e segue il seguente flusso:

Recupera i dati da ciascun campo di input.

Se presente, valida e converte il prezzo massimo in double.

Esegue il metodo GestoreFile.cercaRistoranti(...) passando i filtri raccolti.

Pulisce il resultsPanel e popola i risultati:

Se vuoto, mostra un messaggio.

Altrimenti, per ogni ristorante genera un pulsante con nome, città e tipo cucina.

Cliccando su un pulsante si passa al pannello di dettaglio (RestaurantDetailPanel) con i dati del ristorante selezionato.

11.5 Metodo refresh()

Questo metodo viene chiamato dal frame principale per resettare lo stato del pannello:

Pulisce tutti i campi di input.

Deseleziona le checkbox.

Rimuove tutti i risultati visibili dal pannello dei risultati.

Questo assicura che, ad ogni nuovo accesso al pannello, lo stato sia pulito e pronto per una nuova ricerca.

11.6 Dipendenze

Classe	Scopo
theknife.GestoreFile	Ricerca dei ristoranti nel file CSV tramite filtri.
theknife.Ristorante	Modello dati dei ristoranti, usato per visualizzare i risultati.
theknife.gui.FancyFrame	Permette l'accesso al pannello di dettaglio e gestione delle card.
GradientPanel	Pannello personalizzato con sfondo sfumato.

11.7 Considerazioni progettuali

La classe adotta un'interfaccia utente semplificata e responsiva. L'uso del BoxLayout verticale nel pannello dei risultati permette di visualizzare dinamicamente un numero arbitrario di risultati.

L'approccio modulare favorisce riutilizzo e aggiornamento separato dei criteri di ricerca e del risultato.

La validazione del campo prezzo evita errori di parsing, migliorando la robustezza dell'interfaccia.

12.0 - Classe RistorantiPanel – Gestione dei ristoranti del ristorante

12.1 Descrizione generale

La classe RistorantiPanel rappresenta l'interfaccia grafica dedicata ai ristoratori per visualizzare e gestire i propri ristoranti. È integrata nel frame principale FancyFrame e fornisce un pannello interattivo con funzionalità per aggiungere nuovi ristoranti e accedere ai dettagli di quelli esistenti.

Estende la classe GradientPanel per fornire uno sfondo sfumato e adotta un layout verticale per elencare dinamicamente i ristoranti registrati dal ristorante attualmente loggato.

12.2 Obiettivi funzionali

Visualizzare i ristoranti creati dal ristorante attualmente loggato.

Permettere l'accesso al dettaglio di ciascun ristorante.

Fornire un'interfaccia per aggiungere un nuovo ristorante con tutti i campi richiesti.

Validare i campi numerici inseriti (coordinate geografiche, prezzo).

Salvare i nuovi ristoranti nel file ristoranti.csv.

12.3 Componenti principali

Componente	Tipo	Funzione
resultsPanel	JPanel	Contenitore verticale dei ristoranti registrati.
btnAggiungi	JButton	Pulsante per l'aggiunta di un nuovo ristorante.
JScrollPane	JScrollPane	Contenitore scrollabile per la lista dei ristoranti.

La disposizione dei componenti è suddivisa in tre aree:

North: intestazione con titolo.

Center: pannello scrollabile dei ristoranti.

South: pulsante di aggiunta.

12.4 Metodo refreshData()

Questo metodo aggiorna dinamicamente la lista dei ristoranti mostrati nel pannello:

Rimuove tutti i componenti dal pannello dei risultati.

Controlla se l'utente corrente è un ristorante.

Carica tutti i ristoranti da file tramite GestoreFile.caricaRistoranti.

Filtra quelli il cui campo proprietario corrisponde allo username dell'utente corrente.

Se non presenti, mostra un messaggio; altrimenti genera un pulsante per ciascun ristorante.

Ogni pulsante consente l'accesso al RestaurantDetailPanel.

12.5 Metodo mostraDialogNuovoRistorante()

Gestisce l'interfaccia di inserimento di un nuovo ristorante tramite JOptionPane. I campi previsti sono:

Campo	Tipo	Note
Nome	JTextField	Nome del ristorante.
Nazione	JTextField	Default: "Italia".
Città	JTextField	Città di ubicazione.
Indirizzo	JTextField	Indirizzo completo.
Latitudine	JTextField	Valore di tipo double.
Longitudine	JTextField	Valore di tipo double.
Prezzo	JTextField	Fascia di prezzo media.
Delivery	JCheckBox	Disponibilità del servizio.
Prenotazione	JCheckBox	Possibilità di prenotare un tavolo.
Tipo Cucina	JTextField	Tipo di cucina, default: "Italiana".

Se il dialog viene confermato:
I valori numerici vengono validati.
Viene creato un oggetto Ristorante.
Il ristorante viene salvato con `GestoreFile.aggiungiRistorante()`.
Viene ricaricata la lista dei ristoranti nel pannello tramite `refreshData()`.
Se i valori numerici non sono validi, viene mostrato un messaggio di errore.

12.6 Dipendenze

Classe/Modulo	Funzione
GestoreFile	Caricamento e salvataggio dei ristoranti su file ristoranti.csv.
Ristorante	Modello dati utilizzato per rappresentare i ristoranti.
FancyFrame	Permette accesso all'utente corrente e gestione delle viste a schede (card).
GradientPanel	Pannello base con sfondo sfumato, estensione personalizzata di JPanel.
RestaurantDetailPanel	Pannello richiamato per mostrare i dettagli del ristorante selezionato.

12.7 Considerazioni progettuali

La struttura modulare della classe consente un facile riutilizzo e integrazione nell'interfaccia principale. L'interazione è progettata per essere semplice e immediata per il ristoratore. L'adozione di `BoxLayout` rende la visualizzazione della lista verticale e scalabile. L'aggiunta dinamica dei pulsanti consente un'interazione diretta con i ristoranti registrati. Il sistema di validazione numerica contribuisce alla robustezza della funzionalità di inserimento.

13.0 - Classe RestaurantDetailPanel – Visualizzazione dettagliata del ristorante

13.1 Descrizione generale

La classe RestaurantDetailPanel rappresenta il pannello dedicato alla visualizzazione dettagliata di un singolo ristorante. Estende GradientPanel per includere uno sfondo sfumato ed è utilizzata in combinazione con FancyFrame come una delle "card" del layout principale.

La classe mostra tutti i dati salienti del ristorante, la media delle recensioni ricevute, e l'elenco completo delle recensioni associate, incluse eventuali risposte da parte del ristorante.

13.2 Obiettivi funzionali

Visualizzare in modo completo le informazioni di un ristorante selezionato.

Mostrare la media delle valutazioni in stelle calcolata dinamicamente.

Presentare tutte le recensioni associate, inclusi eventuali commenti di risposta.

Consentire il ritorno alla schermata di ricerca tramite pulsante.

13.3 Componenti GUI

Componente	Tipo	Funzione
textArea	JTextArea	Visualizzazione delle informazioni testuali in formato monospazio.
btnIndietro	JButton	Pulsante per tornare alla schermata di ricerca.

Il layout del pannello è suddiviso in:

North: intestazione con titolo.

Center: JScrollPane contenente textArea.

South: pannello contenente il pulsante di ritorno.

13.4 Metodo setRistorante(Ristorante r)

Permette di impostare il ristorante da visualizzare. Dopo l'assegnazione, chiama refreshDetails() per aggiornare il contenuto della textArea.

13.5 Metodo refreshDetails()

Costruisce dinamicamente il contenuto da mostrare all'utente. Le operazioni effettuate includono:

Verifica se è presente un ristorante corrente.

Recupera e stampa:

Nome, città, nazione, indirizzo.

Fascia di prezzo, disponibilità di delivery e prenotazione.

Tipo di cucina e nome del proprietario.

Calcola la media delle recensioni utilizzando il metodo GestoreFile.calcolaMediaStelle(...).

Carica tutte le recensioni dal file recensioni.csv.

Filtra e stampa solo quelle relative al ristorante selezionato, con:

-Username dell'autore.

-Numero di stelle assegnate.

-Testo della recensione.

-Eventuale risposta del ristorante.

La textArea viene infine aggiornata con il testo completo costruito dinamicamente tramite un StringBuilder.

13.6 Dipendenze

Classe	Funzione
FancyFrame	Gestione delle card e accesso all' interfaccia principale.
Ristorante	Modello dati del ristorante selezionato.
Recensione	Modello dati per recensioni.
GestoreFile	Caricamento recensioni e calcolo media voti.
GradientPanel	Pannello base con sfondo gradiente, usato per aspetto grafico.

13.7 Considerazioni progettuali

Il design di RestaurantDetailPanel è puramente informativo: non modifica dati ma li presenta in modo leggibile e strutturato. Utilizza un font Monospaced per rendere il testo facilmente leggibile e allineato.

La scelta di rappresentare tutti i dati nel componente JTextArea semplifica l'implementazione e migliora la performance per il rendering di contenuti testuali. La struttura di output è leggibile, con un chiaro separatore per la sezione delle recensioni.

Il ritorno alla ricerca è gestito dal pulsante Torna Indietro, che richiama la card CARD_SEARCH nel FancyFrame.

14.0 - Classe RecensioniPanel – Gestione delle recensioni utenti

14.1 Descrizione generale

La classe RecensioniPanel è un pannello grafico del sistema che permette agli utenti di gestire le recensioni dei ristoranti. Estende GradientPanel per fornire un'estetica a sfondo sfumato e si integra con la finestra principale FancyFrame tramite sistema di navigazione a card.

Il pannello consente operazioni distinte in base al ruolo dell'utente:

I clienti possono inserire, modificare ed eliminare le proprie recensioni.

I ristoratori possono rispondere alle recensioni ricevute.

14.2 Obiettivi funzionali

Visualizzare l'elenco completo delle recensioni registrate.

Consentire al cliente autenticato la gestione delle proprie recensioni.

Consentire al ristoratore autenticato di rispondere alle recensioni dei propri ristoranti.

Mantenere aggiornato il file recensioni.csv in lettura e scrittura.

3.18.3 Componenti GUI

Componente	Tipo	Funzione
textArea	JTextArea	Visualizzazione delle recensioni in formato testuale.
btnNuova	JButton	Inserimento di una nuova recensione da parte di un cliente.
btnModifica	JButton	Modifica di una recensione esistente da parte del cliente.
btnElimina	JButton	Eliminazione di una recensione da parte del cliente.
btnRispondi	JButton	Inserimento risposta a una recensione da parte del ristoratore.

14.4 Metodo refreshData()

Questo metodo viene invocato per aggiornare la visualizzazione nella textArea e carica tutte le recensioni dal file data/recensioni.csv.

Se l'utente è autenticato, viene mostrato il suo username per ogni recensione.

Altrimenti, l'autore è visualizzato come Anonimo.

Vengono riportate anche le risposte fornite dai ristoratori, se presenti.

14.5 Metodo nuovaRecensione()

Permette a un cliente autenticato di scrivere una nuova recensione. Il metodo:

Verifica che l'utente sia autenticato e sia un cliente.

Mostra un form di inserimento con:

Scelta del ristorante.

Valutazione in stelle (1–5).

Campo di testo per il commento.

Aggiunge l'oggetto Recensione al file CSV.

14.6 Metodo modificaRecensione()

Permette al cliente autenticato di modificare una propria recensione esistente:

Filtra tutte le recensioni dell'utente corrente.

Presenta una selezione per scegliere quale recensione modificare.

Permette l'inserimento di:

Nuovo testo.

Nuovo numero di stelle.

Utilizza `GestureFile.modificaRecensione(...)` per aggiornare il file.

14.7 Metodo eliminaRecensione()

Consente al cliente di eliminare una delle sue recensioni:

Filtra le recensioni scritte dall'utente autenticato.

Mostra un menu per scegliere quale recensione eliminare.

Richiama `GestoreFile.eliminaRecensione(...)`.

14.8 Metodo rispondiRecensione()

Disponibile solo ai ristoratori autenticati, consente di rispondere alle recensioni:

Identifica tutti i ristoranti il cui proprietario è l'utente corrente.

Filtra le recensioni relative a quei ristoranti.

Mostra un'interfaccia per selezionare la recensione e inserire la risposta.

Aggiorna il campo risposta nella lista e risalva l'intero file tramite `GestoreFile.salvaRecensioni(...)`.

14.9 Considerazioni progettuali

Le operazioni sono protette da verifica di autenticazione e ruolo.

Tutti gli input vengono raccolti tramite finestre modali (`JOptionPane`) per garantire un'interazione utente semplice ma efficace.

Il file `recensioni.csv` funge da sorgente di verità persistente, sempre aggiornato dopo ogni operazione.

Il testo visualizzato nel pannello è impaginato con font `Monospaced` per facilitare la leggibilità.

14.10 Dipendenze

Classe	Funzione
<code>GestoreFile</code>	Caricamento e salvataggio di recensioni e ristoranti.
<code>FancyFrame</code>	Contesto principale dell' interfaccia e riferimento all' utente loggato.
<code>Recensione</code>	Modello dati per ogni recensione.
<code>Ristorante</code>	Verifica proprietà nel caso del ristoratore.
<code>Utente</code>	Identificazione ruolo e proprietà logiche per accesso.

15.0 - Classe PreferitiPanel – Gestione dei ristoranti preferiti del cliente

15.1 Descrizione generale

La classe PreferitiPanel rappresenta un componente dell'interfaccia grafica (GUI) utilizzato per la gestione dei ristoranti preferiti da parte dell'utente con ruolo cliente. Estende GradientPanel, offrendo un'interfaccia moderna con sfondo sfumato, e fa parte del layout a schede della FancyFrame.

Permette agli utenti clienti di:

Visualizzare la lista dei propri ristoranti preferiti.

Aggiungere un ristorante alla lista dei preferiti.

Rimuovere un ristorante dalla lista.

15.2 Obiettivi funzionali

Consentire al cliente loggato di mantenere una lista personalizzata di ristoranti preferiti.

Permettere l'aggiunta e la rimozione tramite interfacce semplici e immediate (JOptionPane).

Sincronizzare la lista con il file preferiti.csv.

15.3 Componenti GUI

Componente	Tipo	Descrizione
textArea	JTextArea	Visualizza i nomi dei ristoranti preferiti del cliente.
btnAggiungi	JButton	Apri una finestra di selezione per aggiungere un ristorante ai preferiti.
btnRimuovi	JButton	Apri una finestra di selezione per rimuovere un ristorante dai preferiti.

Il layout principale è un BorderLayout, suddiviso in:

Nord: Intestazione con titolo.

Centro: Area di testo con l'elenco.

Sud: Pulsanti per le azioni.

15.4 Metodo refreshData()

Aggiorna il contenuto di textArea con i dati correnti dal file preferiti.csv.

Logica:

Verifica se l'utente è autenticato.

Verifica che l'utente sia un cliente.

Carica la lista dei preferiti per lo username attuale.

Mostra i nomi dei ristoranti preferiti.

Se l'utente è un ristoratore o non è loggato, mostra un messaggio informativo nel pannello.

15.5 Metodo aggiungiPreferito()

Permette a un cliente di aggiungere un ristorante alla lista dei preferiti:

Dettagli:

Verifica l'autenticazione e il ruolo.

Carica la lista di tutti i ristoranti disponibili da ristoranti.csv.

Mostra un JComboBox con i nomi.

Salva la selezione nel file preferiti.csv tramite GestoreFile.aggiungiPreferito(...).

Aggiorna l'interfaccia tramite refreshData().

15.6 Metodo rimuoviPreferito()

Permette al cliente di rimuovere un ristorante dalla sua lista di preferiti:

Dettagli:

Verifica l'autenticazione e ruolo.

Carica la lista dei preferiti associata allo username.

Se la lista è vuota, mostra un messaggio informativo.

Altrimenti, consente di selezionare un ristorante da rimuovere.

L'elemento viene eliminato dal file CSV tramite `GestoreFile.rimuoviPreferito(...)`.

Infine, aggiorna l'interfaccia con `refreshData()`.

15.7 Dipendenze

Classe	Funzione
<code>GestoreFile</code>	Lettura/scrittura dei file CSV per ristoranti e preferiti.
<code>FancyFrame</code>	Contesto principale della GUI, contenente l'utente loggato.
<code>Utente</code>	Modello dati dell'utente corrente, usato per filtrare i preferiti.
<code>Ristorante</code>	Modello dati per mostrare i nomi disponibili da selezionare.

15.8 Considerazioni progettuali

Tutti gli accessi e modifiche ai dati persistenti sono centralizzati in `GestoreFile`, garantendo separazione tra logica di presentazione (GUI) e logica dati.

Il pannello mostra messaggi chiari e contestuali per evitare operazioni da utenti non autorizzati.

L'interfaccia si adatta in tempo reale alle modifiche senza dover riavviare la finestra (`refreshData()`).

Le operazioni CRUD sui preferiti sono realizzate in modo intuitivo tramite finestre di dialogo modali (`JOptionPane`).

16.0 - Classe HomePanel – Pannello di benvenuto

16.1 Descrizione generale

La classe HomePanel rappresenta la pagina iniziale (home) dell'applicazione TheKnife, visualizzata all'avvio dell'interfaccia o al logout dell'utente. Si tratta di un semplice pannello informativo che guida l'utente alla scelta tra login o registrazione.

Estende la classe GradientPanel, fornendo uno sfondo sfumato dal colore chiaro.

16.2 Obiettivi funzionali

Presentare un messaggio di benvenuto chiaro e centralizzato.

Indicare all'utente le azioni possibili (login o registrazione).

Servire come pannello di default, mostrato anche al logout.

16.3 Componenti GUI

Componente	Tipo	Descrizione
lbl	JLabel	Etichetta formattata in HTML con un titolo e un messaggio descrittivo.

Il layout utilizzato è un semplice BorderLayout, con il componente posizionato al centro per massima visibilità.

16.4 Logica costruttiva

L'oggetto JLabel utilizza HTML per applicare uno stile semplice (titolo <h1> e paragrafo).

Il testo viene centrato orizzontalmente (SwingConstants.CENTER) e la dimensione del font è definita per maggiore leggibilità.

I colori di sfondo sono gestiti dal GradientPanel, con una sfumatura tra grigio molto chiaro e leggermente più scuro (RGB(240,240,240) → RGB(220,220,220)).

16.5 Dipendenze

Classe	Funzione
FancyFrame	Frame principale che include HomePanel come una delle schede.
GradientPanel	Classe madre che applica lo sfondo sfumato al pannello.

16.6 Considerazioni progettuali

HomePanel funge da landing page iniziale ed è progettato per essere essenziale e non interattivo.

Il suo utilizzo è previsto all'avvio del programma o quando un utente effettua logout.

La semplicità del layout lo rende facilmente adattabile a futuri miglioramenti (es. aggiunta di immagini, pulsanti o animazioni).